作业 __Week8

20244227004 蔡博凯

2024 年 10 月 25 日

第一题

考虑盒子和球模型，状态集合$Q=\{1,2,3\}$，观测集合V=$\{$红，白$\}$，

转移矩阵$A=\begin{bmatrix}0.5 & 0.2 & 0.3\\ 0.3 & 0.5 & 0.2\\ 0.2 & 0.3 & 0.5\end{bmatrix}$，观测矩阵$B=\begin{bmatrix}0.5 & 0.5\\ 0.4 & 0.6\\ 0.7 & 0.3\end{bmatrix}$，

初始矩阵$\pi=[0.2,\ 0.4,\ 0.4]$。设T=4，O=(红，白，红)，请用后向算法计算$P(O|\lambda)$

解答

1 初始化

将观测序列映射为数字：红对应 0，白对应 1，因此 $O=(0,1,0)$。

$$\beta_T(i)=1 \quad \text{对所有 } i$$

即：

$$\beta_3=[1,1,1]$$

2 递推

使用公式：

$$\beta_t(i)=\sum_{j=1}^{N}A_{ij}B_j(O_{t+1})\beta_{t+1}(j)$$

从 $t=2$ 开始向前递推。

对于 $t=2$，观测 $O_3=$ 红 $=0$，计算 $\beta_2(i)$：

$$\beta_2(1)=0.5\times0.5\times1+0.2\times0.4\times1+0.3\times0.7\times1=0.54$$

$$\beta_2(2)=0.3\times0.5\times1+0.5\times0.4\times1+0.2\times0.7\times1=0.49$$

$$\beta_2(3)=0.2\times0.5\times1+0.3\times0.4\times1+0.5\times0.7\times1=0.57$$

因此，$\beta_2=[0.54,0.49,0.57]$。

对于 $t=1$，观测 $O_2=$ 白 $=1$，计算 $\beta_1(i)$：

$$\beta_1(1)=0.5\times0.5\times0.54+0.2\times0.6\times0.49+0.3\times0.3\times0.57=0.2451$$

$$\beta_1(2)=0.3\times0.5\times0.54+0.5\times0.6\times0.49+0.2\times0.3\times0.57=0.2622$$

$$\beta_1(3)=0.2\times0.5\times0.54+0.3\times0.6\times0.49+0.5\times0.3\times0.57=0.2277$$

因此，$\beta_1=[0.2451,0.2622,0.2277]$。

3 计算 $P(O|\lambda)$

根据公式：

$$P(O|\lambda)=\sum_{i=1}^{N}\pi_i B_i(O_1)\beta_1(i)$$

其中 $O_1=$ 红 $=0$，具体计算为：

$$P(O|\lambda)=0.2\times0.5\times0.2451+0.4\times0.4\times0.2622+0.4\times0.7\times0.2277$$

计算得：

$$P(O|\lambda)=0.130218$$

4 结论

观测序列 $O=$（红，白，红）出现的概率 $P(O|\lambda)$ 为 0.130218。

第二题

考虑盒子和球模型，状态集合$Q=\{1,2,3\}$，观测集合V={红，白}，

转移矩阵$A=\begin{bmatrix}0.5 & 0.1 & 0.4\\ 0.3 & 0.5 & 0.2\\ 0.2 & 0.2 & 0.6\end{bmatrix}$，观测矩阵$B=\begin{bmatrix}0.5 & 0.5\\ 0.4 & 0.6\\ 0.7 & 0.3\end{bmatrix}$，

初始矩阵$\pi=[0.2,\ 0.3,\ 0.5]$。设T=8，O=(红，白，红，红，白，红，白，白)，

请用前-后向混合算法计算 $P(i_4=q_3|O,\lambda)$。

解答

将观测序列映射为数字：红对应 0，白对应 1，因此 $O=(0,1,0,0,1,0,1,1)$。

1. 前向算法计算

1.1 初始化

初始化公式如下

$$\alpha_1(i)=\pi_i\cdot B_j(O_1)$$

代入数据可得

$$\alpha_1=[0.2\times0.5,0.3\times0.4,0.5\times0.7]=[0.10,0.12,0.35]$$

1.2 递推

递推公式如下

$$\alpha_t(i)=\left(\sum_{j=1}^{N}\alpha_{t-1}(j)\cdot A_{ji}\right)\cdot B_j(O_t)$$

从 t=2 开始向后递推

对于 $t=2$，观测 $O_2=$白 $=1$，计算 $\alpha_2(i)$：

$$\alpha_2(1)=(0.10\times0.5+0.12\times0.3+0.35\times0.2)\times0.5=0.078$$

$$\alpha_2(2)=(0.10\times0.1+0.12\times0.5+0.35\times0.2)\times0.6=0.084$$

$$\alpha_2(3)=(0.10\times0.4+0.12\times0.2+0.35\times0.6)\times0.3=0.0822$$

因此，$\alpha_2 = [0.078, 0.084, 0.0822]$。

同理，继续计算 t=3,4,...,8 时的 $\alpha_t(i)$

最后得到描述 $\alpha_t(i)$ 的矩阵

$$\alpha = \begin{bmatrix} 0.1 & 0.12 & 0.35 \\ 0.078 & 0.084 & 0.0822 \\ 0.04032 & 0.026496 & 0.068124 \\ 0.0208668 & 0.01236192 & 0.04361112 \\ 0.0114321 & 0.01019392 & 0.01109573 \\ 0.00549669 & 0.00338373 & 0.00928834 \\ 0.00281056 & 0.00245952 & 0.00253453 \\ 0.00132502 & 0.00121063 & 0.00094105 \end{bmatrix}$$

2. 后向算法计算

2.1 初始化

$$\beta_T(i) = 1 \quad \text{对所有 } i$$

即:

$$\beta_8 = [1, 1, 1]$$

2.2 递推

递推公式如下

$$\beta_t(i) = \sum_{j=1}^{N} A_{ij} B_j(O_{t+1}) \beta_{t+1}(j)$$

从 $t = 7$ 开始向前递推。

对于 $t = 7$，观测 $O_8 =$ 白 $= 1$，计算 $\beta_7(i)$:

$$\beta_7(1) = 0.5 \times 0.5 \times 1 + 0.1 \times 0.6 \times 1 + 0.4 \times 0.3 \times 1 = 0.43$$

$$\beta_7(2) = 0.3 \times 0.5 \times 1 + 0.5 \times 0.6 \times 1 + 0.2 \times 0.3 \times 1 = 0.51$$

$$\beta_7(3) = 0.2 \times 0.5 \times 1 + 0.2 \times 0.6 \times 1 + 0.6 \times 0.3 \times 1 = 0.40$$

因此，$\beta_7 = [0.43, 0.51, 0.40]$。

同理，继续计算 t=6,5,...,1 时的 $\beta_t(i)$

最后得到描述 $\beta_t(i)$ 的矩阵

$$\beta = \begin{bmatrix} 0.00632569 & 0.00684706 & 0.00577855 \\ 0.01482964 & 0.01227214 & 0.01568294 \\ 0.02556442 & 0.02343448 & 0.02678985 \\ 0.04586531 & 0.05280909 & 0.04280618 \\ 0.105521 & 0.100883 & 0.111934 \\ 0.1861 & 0.2415 & 0.1762 \\ 0.43 & 0.51 & 0.4 \\ 1 & 1 & 1 \end{bmatrix}$$

3 计算概率 $P(i_4 = q_3|O, \lambda)$

3.1 $P(i_t = q_j|O, \lambda)$ **求解公式**

矩阵下标从 1 开始计数

$$P(i_t = q_j|O, \lambda) = \frac{\alpha[t, j] \cdot \beta[t, j]}{\sum_k \alpha[t, k] \cdot \beta[t, k]}$$

3.2 确定时间步

计算时间步 $t = 4$ 的状态 3 的概率。

3.3 计算过程

分子的计算为：

$$\text{numerator} = \alpha[4, 3] \cdot \beta[4, 3]$$

分母的计算为：

$$\text{denominator} = \sum_j \alpha[4, j] \cdot \beta[4, j]$$

最终概率为：

$$P(i_4 = q_3|O, \lambda) = \frac{\text{numerator}}{\text{denominator}} \approx 0.537$$

4 代码验证

编写代码验证计算结果，可在附录查看完整代码，请参考 前后向混合算法。代码运行结果如下

```
Alpha (前向概率):
[[0.1        0.12       0.35      ]
 [0.078      0.084      0.0822    ]
 [0.04032    0.026496   0.068124  ]
 [0.0208668  0.01236192 0.04361112]
 [0.0114321  0.01019392 0.01109573]
 [0.00549669 0.00338373 0.00928834]
 [0.00281056 0.00245952 0.00253453]
 [0.00132502 0.00121063 0.00094105]]

Beta (后向概率):
[[0.00632569 0.00684706 0.00577855]
 [0.01482964 0.01227214 0.01568294]
 [0.02556442 0.02343448 0.02678985]
 [0.04586531 0.05280909 0.04280618]
 [0.105521   0.100883   0.111934  ]
 [0.1861     0.2415     0.1762    ]
 [0.43       0.51       0.4       ]
 [1.         1.         1.        ]]

P(i_4 = q_3 | O, lambda): 0.5369518160647323
```

图 1: 运行结果

可以观察到，Alpha 和 Beta 矩阵、$P(i_4 = q_3|O, \lambda)$ 的计算结果均正确。

第三题

词性：代词**p**，名词**n**，动词**v**，形容词**a**

字典：**1**你，**2**我，**3**他，**4**喜欢，**5**讨厌，**6**漂亮的，**7**恐怖的，**8**机器学习，**9**操作系统，**10**数据结构

已知$\pi = [0.3\ \ 0.4\ \ 0.2\ \ 0.1]$

$$\boldsymbol{A} = \begin{bmatrix} 0.1 & 0.2 & 0.5 & 0.2 \\ 0.2 & 0.1 & 0.4 & 0.3 \\ 0.3 & 0.3 & 0.1 & 0.3 \\ 0.4 & 0.4 & 0.05 & 0.15 \end{bmatrix} \quad \boldsymbol{B} = \begin{bmatrix} 0.3 & 0.3 & 0.2 & 0.05 & 0 & 0 & 0 & 0.05 & 0.1 & 0 \\ 0.1 & 0.1 & 0.2 & 0.05 & 0 & 0 & 0 & 0.2 & 0.15 & 0.2 \\ 0 & 0.05 & 0 & 0.3 & 0.4 & 0 & 0.2 & 0.05 & 0 & 0 \\ 0 & 0.05 & 0 & 0.05 & 0 & 0.4 & 0.45 & 0.05 & 0 & 0 \end{bmatrix}$$

请用维特比算法为如下句子标注词性标注：“我/喜欢/机器学习”。

解答

1 问题建模

- 词性集合 $S = [p, n, v, a]$：[代词, 名词, 动词, 形容词]
- 观测序列：我/喜欢/机器学习，字典编号为 $[2, 4, 8]$。
- 初始概率分布：$\pi = [0.3, 0.4, 0.2, 0.1]$。
- 转移概率矩阵：

$$A = \begin{bmatrix} 0.1 & 0.2 & 0.5 & 0.2 \\ 0.2 & 0.1 & 0.4 & 0.3 \\ 0.3 & 0.3 & 0.1 & 0.3 \\ 0.4 & 0.4 & 0.05 & 0.15 \end{bmatrix}$$

- 发射概率矩阵：

$$B = \begin{bmatrix} 0.3 & 0.3 & 0.2 & 0.05 & 0 & 0 & 0 & 0.05 & 0.1 & 0 \\ 0.1 & 0.1 & 0.2 & 0.05 & 0 & 0 & 0 & 0.2 & 0.15 & 0.2 \\ 0 & 0.05 & 0 & 0.3 & 0.4 & 0 & 0.2 & 0.05 & 0 & 0 \\ 0 & 0.05 & 0 & 0.05 & 0 & 0.4 & 0.45 & 0.05 & 0 & 0 \end{bmatrix}$$

- 注：矩阵和列表下标均从 1 开始

2 维特比算法步骤

2.1 初始化

对于第一个观测值“我”（编号为 2），初始化每个词性的概率：

$$\delta_1(i) = \pi(i) \times B[i][2]$$

计算结果如下：

$$\delta_1 = [0.3 \times 0.3, 0.4 \times 0.1, 0.2 \times 0.05, 0.1 \times 0.05] = [0.09, 0.04, 0.01, 0.005]$$

路径矩阵初始状态为 $\psi_1 = [-, -, -, -]$ (-表示无前结点)。

2.2 递归

对于每个后续观测值 $t \geq 2$ 和每个词性 j，计算最大概率和路径来源：

$$\delta_t(j) = \max_{1 \leq i \leq N} (\delta_{t-1}(i) \times A[i][j]) \times B[j][O_t]$$

$$\psi_t(j) = \arg\max_{1 \le i \le N} (\delta_{t-1}(i) \times A[i][j])$$

对于第二个观测值“喜欢”（编号 4）：

$$\delta_2(p) = \max\{0.09 \times 0.1, 0.04 \times 0.2, 0.01 \times 0.3, 0.005 \times 0.4\} \times 0.05 = 0.00045$$

$$\delta_2(n) = \max\{0.09 \times 0.2, 0.04 \times 0.1, 0.01 \times 0.3, 0.005 \times 0.4\} \times 0.05 = 0.0009$$

$$\delta_2(v) = \max\{0.09 \times 0.5, 0.04 \times 0.4, 0.01 \times 0.1, 0.005 \times 0.05\} \times 0.3 = 0.0135$$

$$\delta_2(a) = \max\{0.09 \times 0.2, 0.04 \times 0.3, 0.01 \times 0.3, 0.005 \times 0.15\} \times 0.05 = 0.0009$$

因此：

$$\delta_2 = [0.00045, 0.0009, 0.0135, 0.0009]$$

$$\psi_2 = [p, p, p, p]$$

对于第三个观测值“机器学习”（编号 8）：

$$\delta_3(p) = \max\{0.00045 \times 0.1, 0.0009 \times 0.2, 0.0135 \times 0.3, 0.0009 \times 0.4\} \times 0.05 = 0.0002025$$

$$\delta_3(n) = \max\{0.00045 \times 0.2, 0.0009 \times 0.1, 0.0135 \times 0.3, 0.0009 \times 0.4\} \times 0.2 = 0.00081$$

$$\delta_3(v) = \max\{0.00045 \times 0.5, 0.0009 \times 0.4, 0.0135 \times 0.1, 0.0009 \times 0.05\} \times 0.05 = 0.0000675$$

$$\delta_3(a) = \max\{0.00045 \times 0.2, 0.0009 \times 0.3, 0.0135 \times 0.3, 0.0009 \times 0.15\} \times 0.05 = 0.0002025$$

因此：

$$\delta_3 = [0.0002025\ , 0.00081\ , 0.0000675\ , 0.0002025]$$

$$\psi_3 = [v, v, v, v]$$

2.3 回溯

从最后一个观测值对应的最大概率开始，利用路径矩阵 ψ 从后往前逐步找到最优的词性序列。

概率矩阵

各个时刻的最大概率：

$$\Delta = \begin{bmatrix} \delta_1(p) & \delta_1(n) & \delta_1(v) & \delta_1(a) \\ \delta_2(p) & \delta_2(n) & \delta_2(v) & \delta_2(a) \\ \delta_3(p) & \delta_3(n) & \delta_3(v) & \delta_3(a) \end{bmatrix} = \begin{bmatrix} 0.09 & 0.04 & 0.01 & 0.005 \\ 0.00045 & 0.0009 & 0.0135 & 0.0009 \\ 0.0002025 & 0.00081 & 0.0000675 & 0.0002025 \end{bmatrix}$$

路径矩阵

各个时刻的路径来源：

$$\Psi = \begin{bmatrix} \psi_1(1) & \psi_1(2) & \psi_1(3) & \psi_1(4) \\ \psi_2(1) & \psi_2(2) & \psi_2(3) & \psi_2(4) \\ \psi_3(1) & \psi_3(3) & \psi_3(3) & \psi_3(4) \end{bmatrix} = \begin{bmatrix} - & - & - & - \\ p & p & p & p \\ v & v & v & v \end{bmatrix}$$

最佳路径为 (p, v, n)。词性标注如下表所示。

词	词性
我	代词 p
喜欢	动词 v
机器学习	名词 n

表 1: 词性总结

3 代码验证

编写代码验证计算结果，可在附录查看完整代码，请参考 维特比算法。

代码运行结果如下

```
各个时刻的最大概率矩阵 (Delta):
[[9.000e-02 4.000e-02 1.000e-02 5.000e-03]
 [4.500e-04 9.000e-04 1.350e-02 9.000e-04]
 [2.025e-04 8.100e-04 6.750e-05 2.025e-04]]

各个时刻的路径来源矩阵 (字符映射):
[['-' '-' '-' '-']
 ['p' 'p' 'p' 'p']
 ['v' 'v' 'v' 'v']]

最佳路径对应的词性:
p v n
```

图 2: 运行结果

可以观察到，最大概率矩阵和路径来源矩阵、最佳路径的计算结果均正确。

参考文献

[1] https://datawhalechina.github.io/statistical-learning-method-solutions-manua

[2] https://www.bilibili.com/video/BV14s4y1Z7JQ?spm_id_from=333.788.videopod.episodes&vd_source=f90de220471b49ff28d579f90952801a

[3] 李航. 统计学习方法.

A 前后向混合算法

```python
import numpy as np

def main():
    # 给定的模型数据
    A = np.array([[0.5, 0.1, 0.4],
                  [0.3, 0.5, 0.2],
                  [0.2, 0.2, 0.6]])

    B = np.array([[0.5, 0.5],  # 状态 1 的红色和白色概率
                  [0.4, 0.6],  # 状态 2 的红色和白色概率
                  [0.7, 0.3]]) # 状态 3 的红色和白色概率

    pi = np.array([0.2, 0.3, 0.5])  # 初始概率

    # 观测序列 (红色 = 0, 白色 = 1)
    # O = (红, 白, 红, 红, 白, 红, 白, 白)
    observations = [0, 1, 0, 0, 1, 0, 1, 1]
    T = len(observations)  # 时间步数

    # 初始化前向 (alpha) 和后向 (beta) 矩阵
```

```
21      alpha = np.zeros((T, 3))  # 3 个状态
22      beta = np.zeros((T, 3))   # 3 个状态
23
24      # 前向算法 (alpha 计算)
25      # 步骤 1: 初始化 t=1
26      alpha[0, :] = pi * B[:, observations[0]]
27
28      # 步骤 2: 归纳法 t=2,3,...,T
29      for t in range(1, T):
30          for j in range(3):
31              alpha[t, j] = np.sum(alpha[t-1, :] * A[:, j]) * B[j,
                    observations[t]]
32
33      # 后向算法 (beta 计算)
34      # 步骤 1: 初始化 t=T
35      beta[T-1, :] = 1
36
37      # 步骤 2: 归纳法 t=T-1,T-2,...,1
38      for t in range(T-2, -1, -1):
39          for i in range(3):
40              beta[t, i] = np.sum(A[i, :] * B[:, observations[t
                    +1]] * beta[t+1, :])
41
42      # 打印 alpha 和 beta 矩阵
43      print("Alpha (前向概率):")
44      print(alpha)
45
46      print("\nBeta (后向概率):")
47      print(beta)
48
49      # 计算 P(i_4 = q_3 | O, lambda)
50      t = 3  # 这里 t=3 对应于 i_4, 即 t=4
51      numerator = alpha[t, 2] * beta[t, 2]
```

```
    denominator = np.sum(alpha[t, :] * beta[t, :])

    # 最终概率
    P_i4_q3 = numerator / denominator
    print("\nP(i_4 = q_3 | O, lambda):", P_i4_q3)

if __name__ == '__main__':
    main()
```

B 维特比算法

```
import numpy as np

# 定义词性集合
S = ['p', 'n', 'v', 'a']  # 代词, 名词, 动词, 形容词

# 定义观测序列的字典编号
O = [2, 4, 8]  # 对应"我", "喜欢", "机器学习"

# 初始概率分布
pi = np.array([0.3, 0.4, 0.2, 0.1])

# 转移概率矩阵
A = np.array([
    [0.1, 0.2, 0.5, 0.2],
    [0.2, 0.1, 0.4, 0.3],
    [0.3, 0.3, 0.1, 0.3],
    [0.4, 0.4, 0.05, 0.15]
])

# 发射概率矩阵
B = np.array([
```

```
    [0.3, 0.3, 0.2, 0.05, 0, 0, 0, 0.05, 0.1, 0],
    [0.1, 0.1, 0.2, 0.05, 0, 0, 0, 0.2, 0.15, 0.2],
    [0, 0.05, 0, 0.3, 0.4, 0, 0.2, 0.05, 0, 0],
    [0, 0.05, 0, 0.05, 0, 0.4, 0.45, 0.05, 0, 0]
])

# 维特比算法
def viterbi(O, pi, A, B):
    N = len(S)  # 词性数量
    T = len(O)  # 观测序列长度

    # 初始化 delta 和 psi 矩阵
    delta = np.zeros((T, N))
    psi = np.zeros((T, N), dtype=int)

    # 2.1 初始化
    for i in range(N):
        delta[0][i] = pi[i] * B[i][O[0]-1]  # O[t] 需要减 1, 因为索引从 0 开始

    # 设置第一个路径来源为 ['-', '-', '-', '-']
    psi[0] = -1  # 使用 -1 代表无前结点

    # 2.2 递归
    for t in range(1, T):
        for j in range(N):
            # 计算 delta_t(j)
            delta[t][j] = np.max(delta[t-1] * A[:, j]) * B[j][O[t]-1]
            # 计算 psi_t(j)
            psi[t][j] = np.argmax(delta[t-1] * A[:, j])

    # 2.3 回溯
```

```
53      best_path = np.zeros(T, dtype=int)
54      best_path[-1] = np.argmax(delta[T-1])  # 最后时刻的最大概率
            对应的状态
55      for t in range(T-2, -1, -1):
56          best_path[t] = psi[t+1][best_path[t+1]]
57
58      return best_path, delta, psi
59
60  def map_psi_to_char(psi):
61      char_psi = np.empty(psi.shape, dtype=object)  # 创建一个字符
            数组
62      for t in range(psi.shape[0]):
63          for j in range(psi.shape[1]):
64              if psi[t][j] == -1:  # 对应无前结点
65                  char_psi[t][j] = '-'
66              else:
67                  char_psi[t][j] = S[psi[t][j]]  # 将索引映射为字
                        符
68      return char_psi
69
70  if __name__ == "__main__":
71      # 执行维特比算法
72      best_path, delta, psi = viterbi(O, pi, A, B)
73
74      # 输出结果
75      print("各个时刻的最大概率矩阵 (Delta):")
76      print(delta)
77      '''
78      print("\n各个时刻的路径来源矩阵 (Psi):")
79      print(psi)
80
81      print("\n最佳路径索引:")
82      print(best_path)
```

```
83      '''
84      # 输出最佳路径对应的词性
85      # 将路径来源矩阵映射成字符并输出
86      char_psi = map_psi_to_char(psi)
87      print("\n各个时刻的路径来源矩阵 (字符映射):")
88      print(char_psi)
89
90      print("\n最佳路径对应的词性:")
91      for i in best_path:
92          print(S[i], end=' ')  # 输出对应的词性
```